

Document number: P3382R0
Project: Programming Language C++
Audience: LWGI, LEWG, LWG

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2024-08-28

Coarse clocks and resolutions

“One should always aim at being interesting, rather than exact.”

— Voltaire

I. Motivation

Applications that perform frequent timestamp requests are affected by the CPU cost of reading the clock. If the operation is repeated multiple times, the accumulated cost can have an impact on the overall performance of the application.

Many use cases do not need high resolution clocks. For example resolution of hundreds milliseconds or even seconds is fine for the following cases:

- Generating HTTP Date header. One second resolution is fine.
- Periodic jobs. For example doing a ping once in five minutes does not require high resolution from clock.
- Big timeouts for requests. Requests to analytical databases take minutes, no need in high resolution.
- Statistics. Counting response time percentiles or averages does not require high resolution.

Some of the platforms provide *coarse* timers that are more than 70 times faster than the existing standard library clocks [[benchmark](#)]. Due to such drastic performance difference some code bases to minimize CPU usage already use coarse clocks. Moreover it is a common pattern to try to use a coarse clock first and then fallback to precise clock if the resolution is not enough:

```
bool is_expired(std::chrono::steady_clock::time_point deadline) {  
    auto max_time = coarse_steady_clock() + coarse_steady_clock_resolution();  
    if (max_time < deadline) {  
        return false;  
    }  
  
    return deadline < std::chrono::steady_clock::now()  
}
```

Here are some projects that use coarse clocks and/or resolutions:

- [Spdlog](#)
-  [userver framework](#)
- [libuv](#)
- [YDB](#)
- [NodeJS](#)

To sum up: coarse clocks are much more CPU efficient in some cases. Ability to get the coarse clock resolution makes them useful even in more cases.

II. Design decisions

What to do when there's no coarse clock on the platform?

Implementing a coarse clock by just dropping sub-seconds from a more precise clock is fine and still could save a few CPU cycles due to avoiding computations of sub-seconds.

Why `duration` and `time_point` types are same as in non-coarse clocks?

Precise and coarse clocks could be used together (see example from motivation section). Same duration and timepoint types simplify code writing, as less type conversions is needed. Also clock could be a template parameter so same interfaces make it easier to write generic code that use different types of clocks.

III. Wording

Add to the end of `[time.clock.req]`

A type TC meets the *Cpp17TrivialResolutionClock* requirements if

- TC meets the *Cpp17TrivialClock* requirements,
- member function `resolution()` returns positive duration that is the minimal non zero difference between two invocations of `now()`,
- member function `resolution()` does not throw exceptions.

Adjust `[time.clock.general]`

The types defined in `[time.clock]` meet the *Cpp17TrivialResolutionClock* requirements (`[time.clock.req]`) unless otherwise specified.

Adjust each clock in `[time.clock.*]` by adding a member function `resolution()` after function `now()`:

```
static time_point now() noexcept;  
static duration resolution() noexcept;
```

Add after `[time.clock.hires]`:

Class `coarse_steady_clock` `[time.clock.coarse_steady]`

```
namespace std::chrono {  
    class coarse_steady_clock {  
    public:  
        using rep          = steady_clock::rep;  
        using period        = steady_clock::period;  
        using duration      = steady_clock::duration;  
        using time_point    = steady_clock::time_point;  
        static constexpr bool is_steady = true;
```

```

    static time_point now() noexcept;
    static duration resolution() noexcept;
};
}

```

Objects of class `coarse_steady_clock` represent clocks for which values of `time_point` never decrease as physical time advances and for which values of `time_point` advance at a steady rate relative to real time. That is, the clock may not be adjusted.

Result of `resolution()` is coarser than the resolution of `steady_clock`.

Class `coarse_system_clock` [time.clock.coarse_system]

```

namespace std::chrono {
    class coarse_system_clock {
    public:
        using rep          = system_clock::rep;
        using period        = system_clock::period;
        using duration      = system_clock::duration;
        using time_point    = system_clock::time_point;
        static constexpr bool is_steady = system_clock::is_steady;

        static time_point now() noexcept;
        static duration resolution() noexcept;

        // mapping to/from C type time_t
        static time_t      to_time_t(const time_point& t) noexcept {
            return system_clock::to_time_t(t);
        }
        static time_point from_time_t(time_t t) noexcept {
            return system_clock::from_time_t(t);
        }
    };
}

```

Objects of type `coarse_system_clock` represent wall clock time from the system-wide realtime clock and measure time since 1970-01-01 00:00:00 UTC excluding leap seconds. This measure is commonly referred to as Unix time.

Result of `resolution()` is coarser than the resolution of `system_clock`.

Feature test macro

[Editor's note: Add a new macro in `<version>`, `<chrono>`: `__cpp_lib_chrono_coarse` set to the date of adoption].